

Resolving Core Issue #1331 (const mismatch with defaulted copy constructor)

- [Motivation](#)
- [Analysis](#)
- [Proposed Resolution](#)
- [Proposed Wording](#)
- [Issues Resolved](#)

Motivation

If you have a class with a copy constructor whose parameter type is reference-to-nonconst:

```
struct MyType {  
    MyType(MyType&); // no 'const'  
};
```

and you try to aggregate it in a class that has a defaulted copy constructor with the usual signature:

```
template <typename T>  
struct Wrapper {  
    Wrapper(const Wrapper&) = default;  
    T t;  
};  
  
Wrapper<MyType> var; // fails to instantiate
```

the resulting type is ill-formed, even if it is never used in a context where the copy constructor is required.

The authors believe this is unnecessarily restrictive; one should be able to use `Wrapper<MyType>` as long as one does not try to copy it.

`std::tuple` is an example of a wrapper type which suffers from this problem in popular implementations.

Analysis

The relevant standard wording can be found in 8.4.2 [dcl.fct.def.default] paragraph 1 (emphasis ours):

1. [...] A function that is explicitly defaulted shall:
 - be a special member function,
 - have the **same declared function type** (except for possibly differing ref-qualifiers and except that in the case of a copy constructor or copy assignment operator, the parameter type may be "reference to non-const T", where T is the name of the member function's class) **as if it had been implicitly declared**, and
 - not have default arguments.

In the case of `Wrapper<MyType>`, the implicitly declared copy constructor would have the parameter type `MyType&` per 12.8.1 [class.copy.ctor] paragraph 7, so an explicitly defaulted copy constructor with the parameter type `const MyType&` is ill-formed.

Proposed Resolution

This proposal suggests that if the declared type of an explicitly defaulted function is not the same as if it had been implicitly declared, then it be defined as deleted, rather than being ill-formed.

Proposed Wording

1. [...] A function that is explicitly defaulted shall:
 - be a special member function,
 - ~~have the same declared function type (except for possibly differing ref-qualifiers and except that in the case of a copy constructor or copy assignment operator, the parameter type may be "reference to non-const T", where T is the name of the member function's class) as if it had been implicitly declared, and~~
 - not have default arguments.
2. If an explicitly defaulted function does not have the same declared function type (except for possibly differing ref-qualifiers and except that in the case of a copy constructor or copy assignment operator, the parameter type may be "reference to non-const T", where T is the name of the member function's class) as if it had been implicitly declared, it is defined as deleted.

Issues Resolved

This proposal would resolve Core Issues [#1331](#) and [#1426](#), both of which are in Extension status, and are tracked by corresponding Evolution Issues [#101](#) and [#103](#), respectively.